Daily Scholar Papers Report — 2026-06-14

2026-06-14

Daily Scholar Papers Report — 2026-06-14

Download PDF

Window covered: 2026-06-13 → 2026-06-14 (Google Scholar alerts + user-curated self-emails, last 24 h)

Executive Summary

A type-systems-and-program-analysis day. The Outstanding pick is **Sotiropoulos & Su's error-enumeration framework for type analyzers** (PACMPL OOPSLA 2026) — a clean, foundational reframing: rather than fuzzing well-typed programs and watching for crashes, *enumerate ill-typed programs by construction* (inject type mismatches at all single locations of a well-typed seed) and check whether the analyzer accepts any of them; soundness defects are precisely the accepted ill-typed programs. Two Keeps complement it: **ATTAIN** (arXiv, Xin Xia / ZJU+HZAU) — trace-driven diff exploration with an LLM-in-a-finite-state-tool-loop reasoning over version chains; **F1 93.24%** on 224 CVEs × 25,943 library versions × 128 libraries; **RECON** (arXiv, Towson U.) — LLM-enhanced backward constraint analysis on Android bytecode; **5.8× faster** than symbolic execution at **100% success rate** across 78 scenarios with five LLM backends. **OpenPCC** (arXiv, Zhiqiang Lin / Ohio State) is the Borderline-High pick — a vendor-neutral confidential LLM-serving architecture on commodity TEEs, characterised end-to-end on a Llama-3-8B vLLM workload. The CodeLM-natural-backdoor empirical study (Y. Chen et al., arXiv) is logged as a brief Keep for completeness.

Outstanding: 1 · **Keep:** 3 · **Borderline High-Priority:** 1

Highlighted Papers

#	Title	Authors	Venue	Link
1.1	Enumerating Ill-Typed Programs for Testing Type Analyzers	T. Sotiropoulos, Z. Su	PACMPL OOPSLA 2026	DOI
2.1	ATTAIN: Automated Exploit Failure Analysis through Trace-Driven Diff Analysis	X. Mao, Z. Chen, X. Hu, X. Xia	arXiv 2606.09060	arXiv
2.2	RECON: An LLM-Enhanced Backward	B. Bappah, L. Noureddine, U.	arXiv 2606.10264	arXiv

#	Title	Authors	Venue	Link
	Constraint Analysis Framework	Farooq, A. Ali-Gombe		
2.3	Securing Code Understanding: Detecting Natural Backdoor Vulnerability in Code Language Models	Y. Chen, W. Sun, H. Huang, Y. Xiao, C. Fang, Y. Zhang et al.	arXiv 2606.10846	arXiv
3.1	OpenPCC: Open and Confidential LLM Serving on Commodity TEEs	H. Zhou, S. Zhao, C. Wang, Z. Lin	arXiv 2606.11145	arXiv

---

1. Outstanding

1.1 · TYPE-ANALYZER TESTING · PACMPL OOPSLA 2026 — error enumeration: inject single-location type mismatches into well-typed seeds to synthesise ill-typed programs by construction; any analyzer that accepts one is unsound

1.1 Enumerating Ill-Typed Programs for Testing Type Analyzers — Sotiropoulos, Su (ETH Zürich), PACMPL OOPSLA 2026

Authors: Thodoris Sotiropoulos, Zhendong Su (ETH Zürich). **Venue:** *Proceedings of the ACM on Programming Languages*, OOPSLA 2026. DOI: 10.1145/3808320. **License:** ACM — no figure embedding; abstract-level read.

Problem. Production type analyzers (TypeScript's `tsc`, Flow, mypy, PyRight, the Java/Kotlin/Scala/Rust checkers, the Hindley–Milner front-ends of OCaml/Haskell) ship with millions of LOC of typing rules; soundness bugs — *accepting an ill-typed program* — are the most consequential failures because they propagate through every downstream tool that trusts the type. The dominant testing methodology has been *well-typed* program generation (Csmith-style for the type level, randomised AST synthesis, fuzzing) plus crash/oracle differencing — but those campaigns target the analyzer's robustness, not its soundness, and they miss the failure mode that matters most: the analyzer should *reject* a program that violates the rules.

Approach — error enumeration. Reframe the problem: instead of generating well-typed programs and checking for crashes, generate ill-typed programs *by construction* and check whether the analyzer (incorrectly) accepts them. Given a well-typed seed program P, *systematically inject type mismatches at all possible program locations* — every expression, every binding site, every parameter, every return position — producing a finite family of single-injection ill-typed variants $\{P_i'\}$. Each P_i' is, by construction, *guaranteed* to contain at least one type error; if the analyzer reports no error on P_i', that is a soundness defect, with a minimal localisable witness.

The methodology has three attractions over fuzz-style testing:

- **Ground truth comes free.** A variant P_i' comes with the exact location of the seeded mismatch, so a missed-error report has a precise oracle. No differential testing across compilers is required.
- **The injection space is structured.** By enumerating injection *positions* and *type swaps* drawn from the seed's own type environment, every variant is a near-neighbour of a valid program — the same scaling property that made mutation testing tractable.
- **It composes with existing well-typed corpora.** Any well-typed test suite (TypeScript's `tsc` test/, Flow's `tests/`, the mypy primer, MaJ for Java) becomes a starting set of seeds; the methodology amplifies a corpus of size N into N · (locations × swaps) ill-typed probes.

Why Outstanding (research direction). Soundness testing of static type checkers has long been the harder cousin of completeness testing — completeness is easy to refute (find a well-typed program the analyzer rejects), soundness is hard (you have to *know* the program is ill-typed). Construction-by-injection makes the ground truth trivial. The framing is general enough to lift to any decidable judgement that distinguishes accepted vs. rejected programs (refinement-type checkers, effect systems, ownership/borrow checkers, gradual-type sound-island annotations) — the same machinery applies to any rule-based analyzer where "should reject" is the interesting failure mode. The Su lab has demonstrated this kind of methodology-first reframing repeatedly (Csmith→YARPGen→…); error enumeration looks like the soundness counterpart.

Caveats. Abstract-only read — exact analyzers tested, bug counts, and the precise enumeration semantics for multi-location injections were not extracted; deferred to a re-read when the ACM full text is fetched. Single-location injection may miss compound soundness bugs that require two simultaneous mismatches to escape a flow-sensitive rule. The methodology assumes the *seed* is genuinely well-typed; if the analyzer being tested is the only authority on that, there's a small circularity that the paper presumably addresses by cross-checking against a reference checker.

Closing-line verbatim (from snippet). "We propose error enumeration, a method that aims to discover soundness defects in type analyzers by generating ill-typed programs by construction."

---

2. Keep

2.1 · EXPLOIT FAILURE ANALYSIS · arXiv 2026 — trace-driven diff exploration with an LLM-in-a-finite-state-tool-loop reasons over historical library versions; F1 93.24% on 224 CVEs × 25,943 versions × 128 libraries

2.1 ATTAIN: Automated Exploit Failure Analysis through Trace-Driven Diff Analysis — Mao, Chen, Hu, Xia (Zhejiang University / Huazhong University of Science and Technology), arXiv 2026

Authors: Xinwei Mao, Zirui Chen, Xing Hu, Xin Xia. **Venue:** arXiv:2606.09060, 10 Jun 2026. **License:** arXiv default non-exclusive — no figure embedding.

Problem. Two prevailing approaches to mark CVE-affected library versions both have known failure modes:

- *Exploit-based checks* — replay a known PoC against each candidate version. Sound when it runs, but exploits routinely stop working across versions due to API changes, removed-method renames, dependency drift, build/environment differences — leaving long version chains *unjudged* rather than *judged-vulnerable* or *judged-fixed*.
- *Commit-based methods* (SZZ-style line-blame on the fix commit) — sometimes miss the introducing commit and propagate the wrong label across the version graph, especially for refactoring-heavy fix histories or backported patches.

The cost of getting this right is real: incorrect affected-version ranges cascade into SBOM, SCA, and patch-presence tooling.

Approach — ATTAIN, three modules.

1. **Trace construction.** Execute the exploit on each historical version of the library; capture the execution trace; compute the cross-version *behavioural divergence* — the points at which the same exploit produces different runtime behaviour across versions. The divergence localises *where* a version differs from a known-vulnerable baseline, not whether it's vulnerable.

2. **Diff exploration.** Feed the divergence to an LLM operating in a *finite-state tool loop* — the LLM is constrained to a small set of analysis actions (read diff hunk, ask for callgraph context, inspect commit history, compare with adjacent version) and iterates until it has collected enough vulnerability-relevant diff hunks to make a judgement. The finite-state framing is the key engineering decision: it bounds the LLM's freedom so that the search terminates and is reproducible, while still letting the model decide *which* tool to call next based on what it has seen.

3. **Affected-version judgment.** A separate reasoner ingests the collected diff evidence and decides per-version: vulnerable, fixed, or non-applicable, and emits an affected-version range.

Headline numbers.

- Evaluation set: **224 CVEs, 25,943 library versions, 128 libraries** — order of magnitude larger than typical SZZ-style evaluations.
- **F1-score 93.24%**, beating the commit-based comparator (snippet truncates the exact baseline number; the paper reports the comparison in §5).

Why Keep. Two reasons:

- The "exploit dies in transit, label propagates wrong" failure mode is the single largest source of noise in production CVE/version mapping, and ATTAIN is one of the first methods to explicitly target it with a structured trace-then-diff pipeline rather than yet another commit-classifier.
- The *finite-state LLM tool loop* is a reusable design pattern. Many recent agent-style papers either let the LLM loop unconstrained (slow, expensive, hard to reproduce) or hardcode the entire workflow (no flexibility). A finite-state envelope with LLM-as-action-selector is a Goldilocks design worth borrowing in other security-analysis contexts (patch backporting, vulnerability triage, exploit-derivative classification).

Caveats. Exploit replay requires a runnable PoC and a buildable historical version — both are fragile assumptions. The F1 is computed against an LLM-or-human-labelled ground truth; details of label quality matter and require a re-read. Generalisation to libraries without rich version history (private code, fast-moving experimental projects) is unclear.

2.2 · LLM + STATIC ANALYSIS · arXiv 2026 — backward path discovery on Android bytecode with LLM reasoning; 5.8× faster than symbolic execution at 100% success across 78 scenarios with five LLM backends

2.2 RECON: An LLM-Enhanced Backward Constraint Analysis Framework — Bappah, Noureddine, Farooq, Ali-Gombe (Towson University / UC Riverside), arXiv 2026

Authors: Babangida Bappah, Lamine Noureddine, Umar Farooq, Aisha Ali-Gombe. **Venue:** arXiv:2606.10264, 10 Jun 2026. **License:** arXiv default non-exclusive — no figure embedding.

Problem. Classical symbolic execution gives precise constraints but fails to scale on modern Android applications because of (i) path explosion in the presence of long control-flow chains, (ii) function-modelling requirements for the Android framework, and (iii) loss of semantic intent when reasoning at the bytecode level — variable names, comments, and high-level operations are gone, leaving only the Smali/dex skeleton. In event-driven, framework-heavy environments these limitations compound: the constraints that matter to a malware analyst (e.g. "under what condition does this method send the IMEI off-device?") sit behind layers of framework callbacks.

Approach — RECON.

- *Backward path discovery.* Starting from a target method (e.g. a sink like `SmsManager.sendTextMessage`), walk the call graph backward to one or more application entry points (Activities, BroadcastReceivers, Services), collecting the *method-level control-flow constraints* on the way — branch predicates, parameter constraints, callback ordering.
- *LLM bytecode interpretation.* Hand each accumulated bytecode condition to an LLM and ask for an *interpretable specification* — a natural-language and structured representation of what the condition means in terms of application semantics. The LLM is *augmenting* the analysis with semantic understanding, not replacing the static-analysis backbone.
- *Output.* For each target method, a backward-reaching specification: the set of execution paths from entry points, the constraints along each, and an LLM-elevated semantic explanation.

Headline numbers.

- 78 Android constraint-extraction scenarios; five LLM backends evaluated as the reasoning component.
- **5.8× faster** than traditional symbolic execution (likely angr / Soot-Boomerang baseline; details require a re-read).
- **100% success rate** (every target produced a constraint; the symbolic-execution comparator timed out on a substantial fraction).
- Logical equivalence maintained vs. SE on the cases where both completed.
- Downstream malware-analysis evaluation on 100 samples (results truncated in snippet).

Why Keep. The interesting design is the *direction* of the LLM-static-analysis bridge — most "LLM + static analysis" papers use the LLM to *find candidate sinks* and let SA prove them. RECON inverts this: it uses SA to compute the path constraints precisely (which LLMs are bad at) and uses the LLM to *render*

those constraints into human-meaningful specifications (which SA is bad at). This is a useful template for any reverse-engineering or audit workflow where the final consumer is a human analyst.

Caveats. Android-specific implementation; portability to JVM / native binaries is non-trivial. The "100% success rate" framing is suspicious — the baseline likely timed out on hard cases that RECON solves more easily, but a head-to-head with a competitive SE configured with the same time budget would be more convincing. LLM faithfulness to the underlying bytecode condition needs auditing; a hallucinated specification at this stage poisons downstream consumers.

2.3 · CODELM SECURITY · arXiv 2026 — empirical study of *naturally occurring* backdoors in CodeLMs across 44 scenarios; characterises injected vs natural backdoors at model / parameter level and proposes ScanNBT detection

2.3 Securing Code Understanding: Detecting Natural Backdoor Vulnerability in Code Language Models — Chen, Sun, Huang, Xiao, Fang, Zhang, Xu, Chen, Guo, Lv, Zhang, Chen, Liu, Xu (Nanjing University et al.), arXiv 2026

Authors: Yuchen Chen, Weisong Sun, Haocheng Huang, Yuan Xiao, Chunrong Fang, Yiran Zhang, Tingting Xu, Zhenpeng Chen, An Guo, Peizhuo Lv, Xiaofang Zhang, Zhenyu Chen, Yang Liu, Baowen Xu. **Venue:** arXiv:2606.10846, 10 Jun 2026. **License:** arXiv default non-exclusive — no figure embedding.

Problem. Code Language Models have well-studied susceptibility to *injected* backdoor attacks (data poisoning, trojan triggers). A separate, less-studied phenomenon — *natural backdoors* — has been observed in computer-vision DL models: trigger-like artefacts that emerge from normal training without any malicious injection. Whether CodeLMs exhibit the same phenomenon, and what its operational consequences are for downstream code-intelligence tasks, has not been characterised.

Approach — empirical study, 44 scenarios. Across architectures and code-intelligence tasks (code summarisation, code search, code clone detection, defect detection — the precise mix requires a re-read), the authors:

- Characterise the prevalence of natural backdoor vulnerabilities in normally-trained CodeLMs.
- Compare injected vs natural backdoors at the *model level* (functional behaviour) and *parameter level* (where in the network the trigger lives).
- Study transferability across datasets, model architectures, and shared knowledge (likely tokeniser / pretraining overlap).
- Investigate causes from training data and training procedure.
- Evaluate existing backdoor defences (pre-training / in-training / post-training) on the natural variant.
- Propose **ScanNBT**, a detection method designed for the natural-backdoor case (snippet truncates at proposal; methodology requires a re-read).

Why this is logged as a brief Keep. The natural-backdoor phenomenon is genuinely interesting as a security concern for code-intelligence pipelines, and a thorough 44-scenario empirical mapping has value as a reference. The contribution sits inside a saturated topic (CodeLM-backdoor) and the methodology — empirical characterisation plus a detector — is in the standard playbook, so it does not promote to Outstanding without further evidence of the detector beating prior natural-backdoor defences. Worth a re-read when the methodology section is available.

Caveats. Snippet truncates before headline numbers; defence-evaluation comparator set unknown; ScanNBT details unread.

---

3. Borderline High-Priority

3.1 · CONFIDENTIAL LLM SERVING · arXiv 2026 — vendor-neutral confidential CIS on commodity TEEs; open-source prototype on Llama-3-8B vLLM with end-to-end characterisation that separates framework cost from TEE-hardware cost

3.1 OpenPCC: Open and Confidential LLM Serving on Commodity TEEs — Zhou, Zhao, Wang, Lin (Ohio State University), arXiv 2026

Authors: Haoling Zhou, Shixuan Zhao, Chao Wang, Zhiqiang Lin. **Venue:** arXiv:2606.11145, 11 Jun 2026. **License:** arXiv default non-exclusive — no figure embedding.

Problem. Cloud Inference Services (CIS) now process user requests that often contain sensitive personal or enterprise-confidential content. Industry-grade confidential CIS efforts — Apple Private Cloud Compute (PCC), Google Private AI Compute — demonstrate the *feasibility* of strong end-to-end protection but are not deployable by third parties: they rely on proprietary hardware and closed ecosystems. Moreover, the authors argue these systems carry their own design glitches that limit the privacy guarantee.

Approach — OpenPCC.

- *Requirements analysis* — distil the fundamental requirements for a secure-yet-open CIS (attestation, encrypted KV cache, sealed user-data lifecycle, vendor-neutral attestation), explicitly noting where Apple PCC / Google Private AI Compute fall short.
- *Architecture* — TEE-based serving framework using commercially available TEEs (the snippet does not name the vendor mix; the standard candidates are Intel TDX, AMD SEV-SNP, and ARM CCA).
- *Open-source prototype + end-to-end characterisation* — Llama-3 8B running under vLLM, profiled to separate OpenPCC's own overhead from the underlying TEE hardware cost. The separation matters because TEE hardware is a moving target — knowing how much overhead the framework adds on top of the hardware floor is what tells future deployers whether OpenPCC is worth adopting on the next-generation TEE.

Why Borderline-High. Topic is slightly off-axis from the digest's vulnerability-detection / program-analysis core, but two things lift it: (i) the *separation-of-costs* characterisation methodology is reusable for any TEE-based system paper, (ii) the architectural decisions (where to put attestation, how to handle the KV cache) are the canonical decision points that anyone building privacy-preserving ML infrastructure will need to revisit — having a vendor-neutral reference point matters. Bumps below Keep because the systems-security angle is a secondary topic for this digest.

Caveats. Snippet truncates before threat-model and performance overhead numbers. TEE side-channels (cache, microarchitectural, controlled-channel) are the perennial concern with commodity-TEE systems; whether OpenPCC inherits or mitigates them needs the full paper.

Cross-Paper Synthesis

Three of today's five highlighted papers — error enumeration for type analyzers (1.1), ATTAIN (2.1), and RECON (2.2) — share a structural pattern that's becoming this year's defining methodology in automated program analysis: **invert the failure-mode framing**. Sotiropoulos & Su flip "test the analyzer with well-typed programs" to "test it by *constructing* the ill-typed ones it should reject". ATTAIN flips "replay the exploit and trust the result" to "watch what *diverges* across versions and ask an LLM which divergence matters". RECON flips "let the LLM find sinks and let SA prove them" to "let SA compute the constraints and let the LLM render them human-readable". The common move is to identify which step in the conventional pipeline is the *known-hard* one and re-engineer the workflow around producing structured evidence for that step.

Two of the LLM-augmented systems (ATTAIN and RECON) also share a *finite-state envelope* design — the LLM is constrained to a small set of actions and a bounded loop, not given an open-ended planning prompt. That is a quietly important pattern for reproducibility in security research: an LLM that can call any tool any number of times produces results that depend on sampling temperature; an LLM constrained to a finite-state machine with a tool budget produces results that other groups can reproduce. Worth borrowing into any LLM-as-analyst pipeline.

The two backdrop themes — natural backdoors in CodeLMs (2.3) and confidential LLM serving on commodity TEEs (3.1) — both reflect the security community catching up with the LLM-as-infrastructure transition. (2.3) names a hazard that was always going to be there in CodeLMs once they became dependency-graph-scale infrastructure; (3.1) addresses the corresponding deployment-side hazard that processing sensitive content under a non-attested CIS is becoming a compliance issue. Both are early entries in what will likely be an active subfield over the next 12–18 months.

Writing & Rationale Insights

- The Su paper's title verb is *"Enumerating"*, not *"Generating"* — a single-word choice that signals exhaustive injection rather than randomised sampling. When the inversion of a classical pipeline is the core contribution, the title should advertise the inversion verb explicitly; readers scanning a TOC need that signal.
- ATTAIN's writeup pays for the LLM-in-the-loop component by framing it as a *finite-state tool loop* — that bounds the search and makes the result reproducible. The phrasing "finite-state tool loop" is precise and citable; it's a better term than the vaguer "agent loop" that has shown up in many 2025-2026 papers.
- RECON shows up to the literature with *5.8× speedup* and *100% success rate* — the second number invites scepticism because perfect success rates usually mean the comparator timed out. Reporting both the speedup and a per-comparator timeout fraction would make this stronger.
- The CodeLM-natural-backdoor study (2.3) chose a long author list and a 44-scenario sweep; both decisions add to legitimacy but increase the bar for the proposed *ScanNBT* detector — a 14-author empirical study is expected to ship a detector that beats published natural-backdoor defences, not match them.